

TP6 : Responsive Design avec CSS

Adapter un site web aux ecrans d'ordinateur, de tablette et de smartphone

Par Dr YANKAM Y. Florian | IUT-FV, UDs | TP Programmation Web

1. Position du TP dans la progression

Dans le TP4, le mini-site a été amélioré avec du CSS afin de mieux gérer les couleurs, les polices, les espacements et l'apparence générale des pages.

Dans le TP5, la mise en page a été rendue plus professionnelle avec Flexbox. Le site possède maintenant une organisation plus claire avec :

- un en-tête
- un menu de navigation
- une zone principale
- une barre latérale
- un pied de page

Cependant, un problème important reste à corriger : le site peut être agréable sur un ordinateur, mais devenir difficile à utiliser sur un petit écran. Sur smartphone, les colonnes peuvent devenir trop serrées, les images peuvent dépasser, les liens peuvent être trop proches les uns des autres et la lecture peut devenir inconfortable.

Le TP6 introduit donc le responsive design, c'est-à-dire l'ensemble des techniques permettant à un site web de s'adapter automatiquement à la taille de l'écran utilisé.

2. Objectifs pédagogiques

À la fin de ce TP, l'étudiant doit être capable de :

- comprendre l'intérêt du responsive design
- utiliser la balise meta viewport
- utiliser des largeurs flexibles au lieu de dimensions fixes
- rendre les images adaptatives
- utiliser les media queries
- adapter une mise en page Flexbox aux petits écrans
- transformer un menu horizontal en menu vertical sur mobile
- tester le rendu mobile avec les outils du navigateur
- produire un site statique plus proche d'un travail de développeur frontend junior

3. Comprendre le problème d'un site non responsive

Un site non responsive est un site conçu uniquement pour une taille d'écran donnée. Les problèmes les plus fréquents sont :

- le texte devient trop petit
- les colonnes restent côte à côte alors qu'il n'y a plus assez d'espace
- les images dépassent la largeur de l'écran
- les boutons et les liens deviennent difficiles à toucher
- l'utilisateur doit faire défiler horizontalement la page
- le menu devient peu lisible

4. Principe du responsive design

Le responsive design repose sur une idee simple : la page doit changer d'organisation selon la largeur disponible.

Sur un ecran large, on peut afficher plusieurs blocs cote a cote. Sur un petit ecran, cette disposition devient moins confortable. Il est donc preferable d'empiler les blocs verticalement.

Le responsive design utilise generalement :

- des largeurs flexibles
- des unites relatives
- des images adaptatives
- des media queries
- des mises en page flexibles comme Flexbox

5. Preparation du dossier de travail

Creer une copie du dossier du TP6 et la renommer : TP6_Responsive_Design

Le dossier doit contenir :

```
TP6_Responsive_Design/  
  index.html  
  presentation.html  
  contact.html  
  style.css  
  images/  
    profil.jpg
```

6. Ajout de la balise meta viewport

Avant d'ecrire les media queries, il faut preparer les pages HTML pour les ecrans mobiles. Dans chaque fichier HTML (index.html, presentation.html et contact.html), ajouter dans la partie <head> :

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">  
<!-- Demande au navigateur d'adapter la largeur de la page a la largeur reelle de  
l'ecran -->
```

Exemple complet de la partie <head> :

```
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Accueil - Mon mini-site</title>  
  <link rel="stylesheet" href="style.css">  
</head>
```

NB; Cette balise est tres importante. Sans elle, le navigateur mobile peut afficher la page comme si elle etait prevue pour un grand ecran, puis la reduire artificiellement.

7. Observation du site avant correction

Avant de modifier le CSS, ouvrir la page index.html dans le navigateur. Reduire progressivement la largeur de la fenetre. Observer les problemes eventuels :

- le menu reste-t-il lisible ? **OUI**
- le contenu principal et la barre laterale restent-ils trop serres ? **NON**
- une image depasse-t-elle la largeur de la page ? **NON**

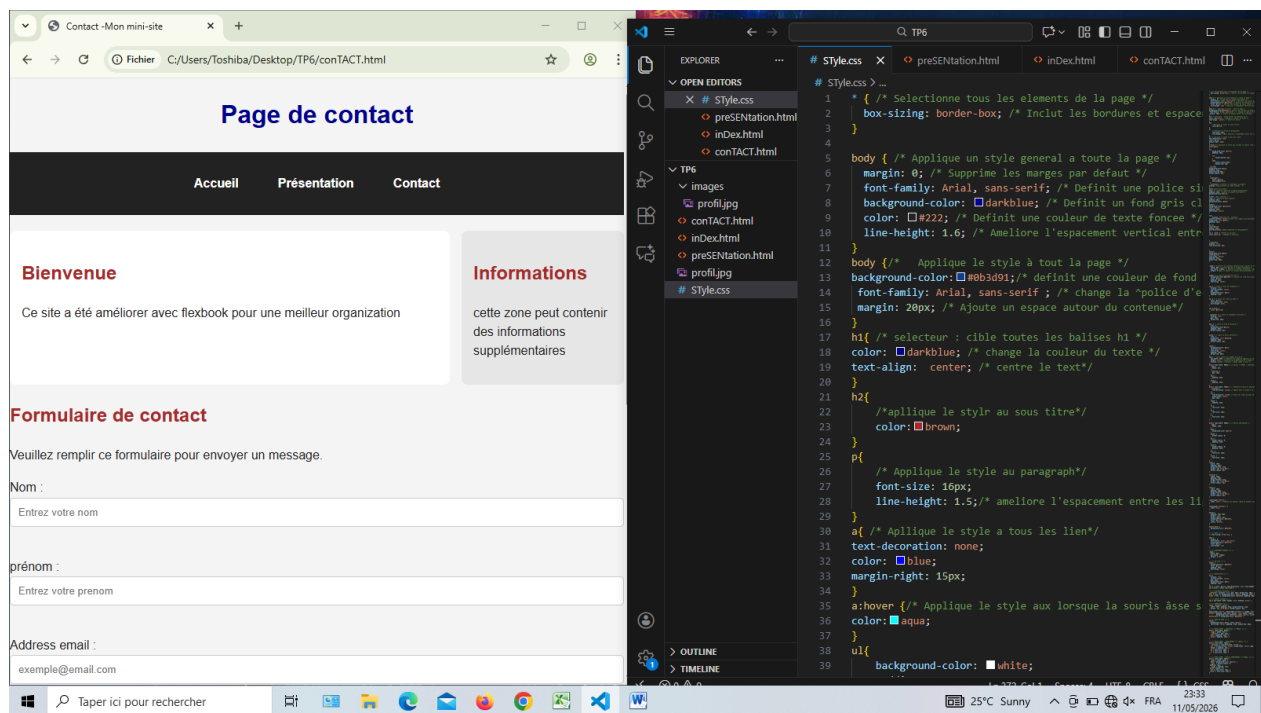
- le texte reste-t-il confortable a lire ? **OUI**
- existe-t-il un defilement horizontal ? **NON**

8. Mise en place d'une base CSS plus propre

Dans le fichier style.css, commencer par verifier ou ajouter les regles suivantes au debut du fichier :

```
* { /* Selectionne tous les elements de la page */
  box-sizing: border-box; /* Inclut les bordures et espacements internes dans la
largeur */
}

body { /* Applique un style general a toute la page */
  margin: 0; /* Supprime les marges par default */
  font-family: Arial, sans-serif; /* Definit une police simple et lisible */
  background-color: #f4f4f4; /* Definit un fond gris clair */
  color: #222; /* Definit une couleur de texte foncée */
  line-height: 1.6; /* Ameliore l'espacement vertical entre les lignes */
}
```



9. Creation d'un conteneur general fluide

Dans un site responsive, il est deconseille de donner une largeur fixe trop rigide a la page. Ajouter dans style.css :

```
.page { /* Definit un conteneur general pour limiter la largeur du site */
  width: 90%; /* Occupe 90% de la largeur disponible */
  max-width: 1100px; /* Empeche le contenu de devenir trop large sur grand ecran */
  margin: 0 auto; /* Centre le contenu horizontalement */
}
```

Dans les fichiers HTML, placer le contenu principal dans une balise <div class="page">.

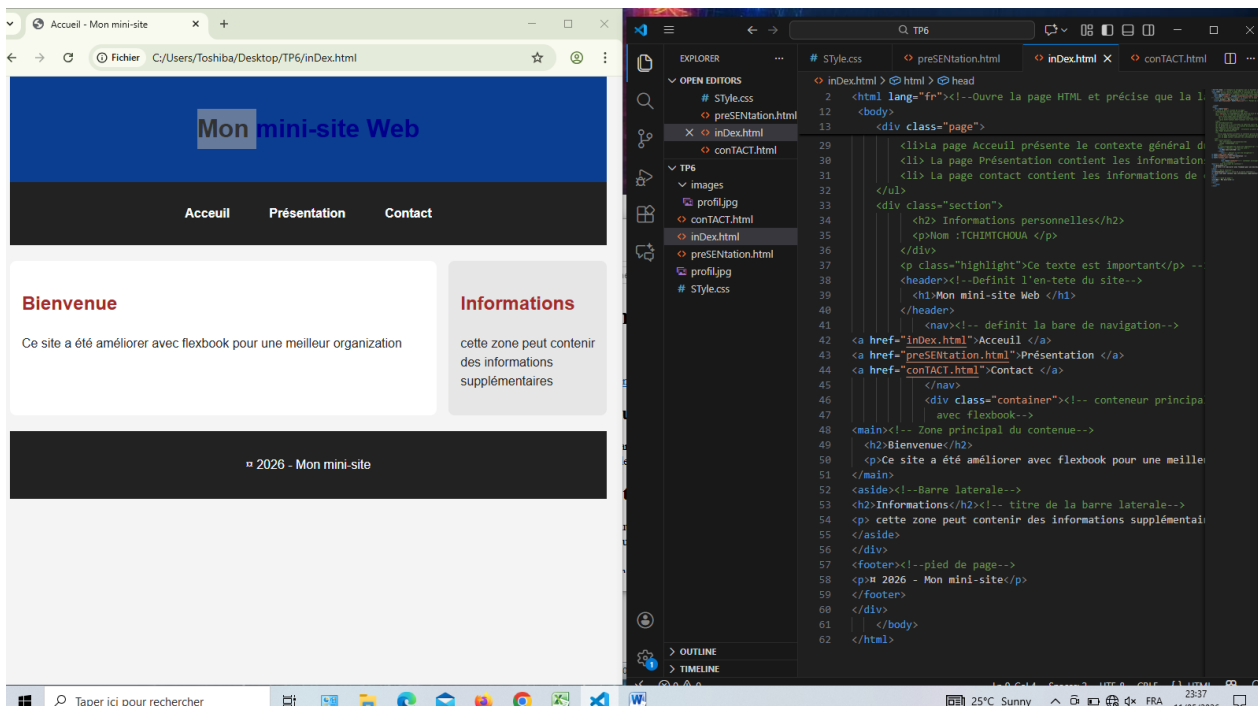
Exemple pour index.html :

```
<body>
  <div class="page">
```

```

<header>
  <h1>Mon mini-site web</h1>
</header>
<nav>
  <a href="index.html">Accueil</a>
  <a href="presentation.html">Presentation</a>
  <a href="contact.html">Contact</a>
</nav>
<div class="container">
  <main>
    <h2>Bienvenue</h2>
    <p>Ce site est progressivement ameliore pour devenir lisible sur ordinateur,
    tablette et smartphone.</p>
  </main>
  <aside>
    <h2>Informations</h2>
    <p>Cette zone contient des informations complementaires.</p>
  </aside>
</div>
<footer>
  <p>© 2026 - Mon mini-site</p>
</footer>
</div>
</body>

```



10. Mise en forme de la structure principale

Ajouter ou corriger les regles suivantes dans style.css :

```

header { /* Cible l'en-tete du site */
  background-color: #0b3d91; /* Couleur de fond bleu fonce */
  color: white;
  padding: 20px;
  text-align: center;
}

nav { /* Cible le menu de navigation */
  display: flex;
  justify-content: center;
  gap: 20px;
  background-color: #222;
  padding: 12px;
}

```

```

nav a { /* Cible les liens du menu */
  color: white;
  text-decoration: none;
  font-weight: bold;
}

nav a:hover {
  color: #ffcc00;
}

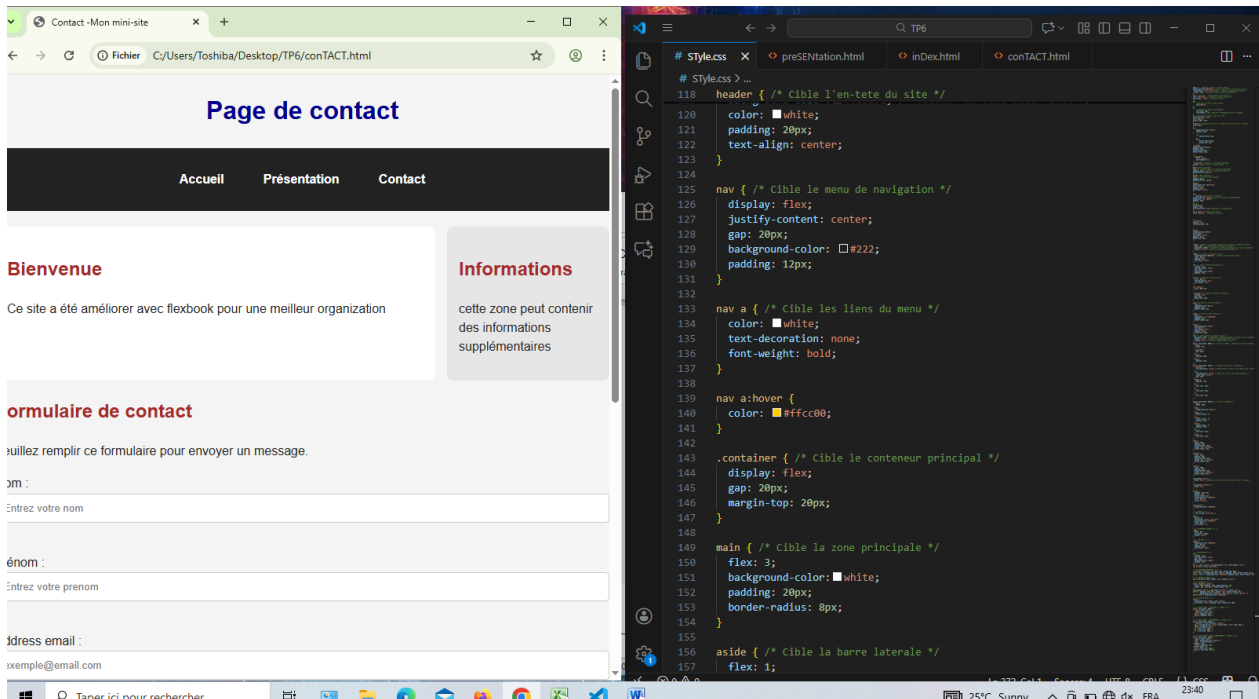
.container { /* Cible le conteneur principal */
  display: flex;
  gap: 20px;
  margin-top: 20px;
}

main { /* Cible la zone principale */
  flex: 3;
  background-color: white;
  padding: 20px;
  border-radius: 8px;
}

aside { /* Cible la barre laterale */
  flex: 1;
  background-color: #e6e6e6;
  padding: 20px;
  border-radius: 8px;
}

footer {
  background-color: #222;
  color: white;
  text-align: center;
  padding: 15px;
  margin-top: 20px;
}

```

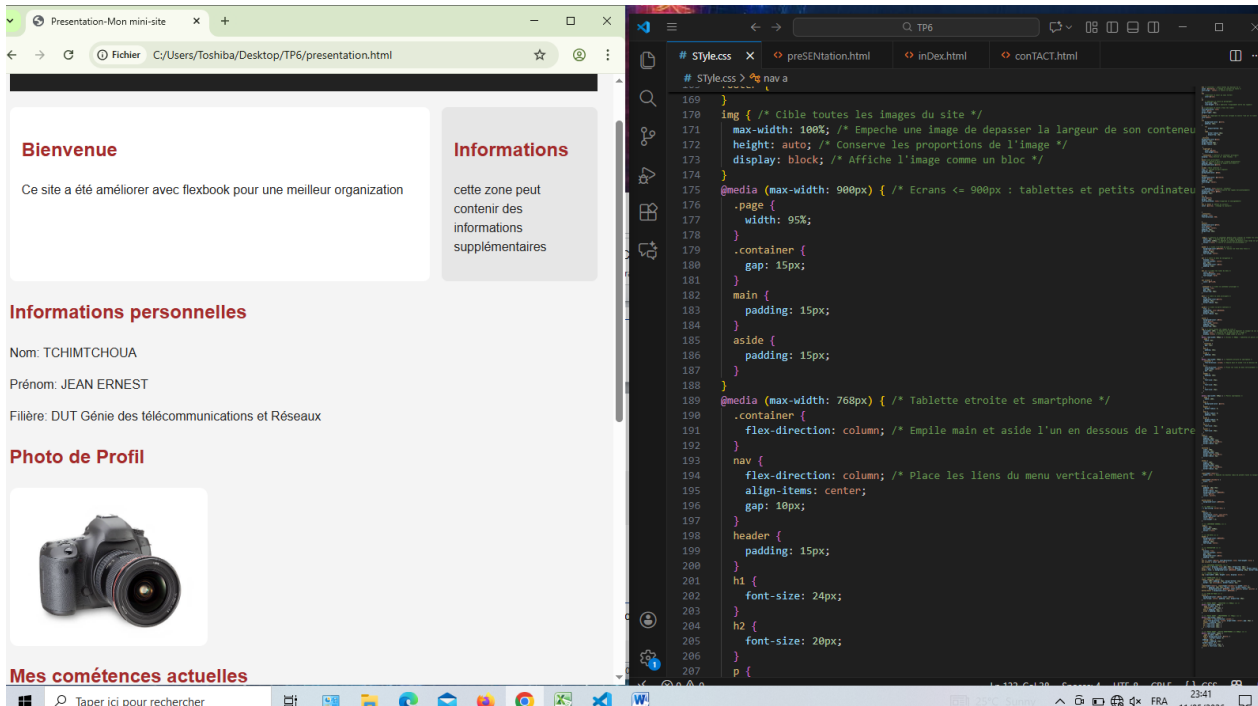


11. Rendre les images fluides

Les images sont souvent la cause de problèmes sur mobile. Ajouter dans style.css :

```
img { /* Cible toutes les images du site */
  max-width: 100%; /* Empêche une image de dépasser la largeur de son conteneur */
  height: auto; /* Conserve les proportions de l'image */
  display: block; /* Affiche l'image comme un bloc */
}
```

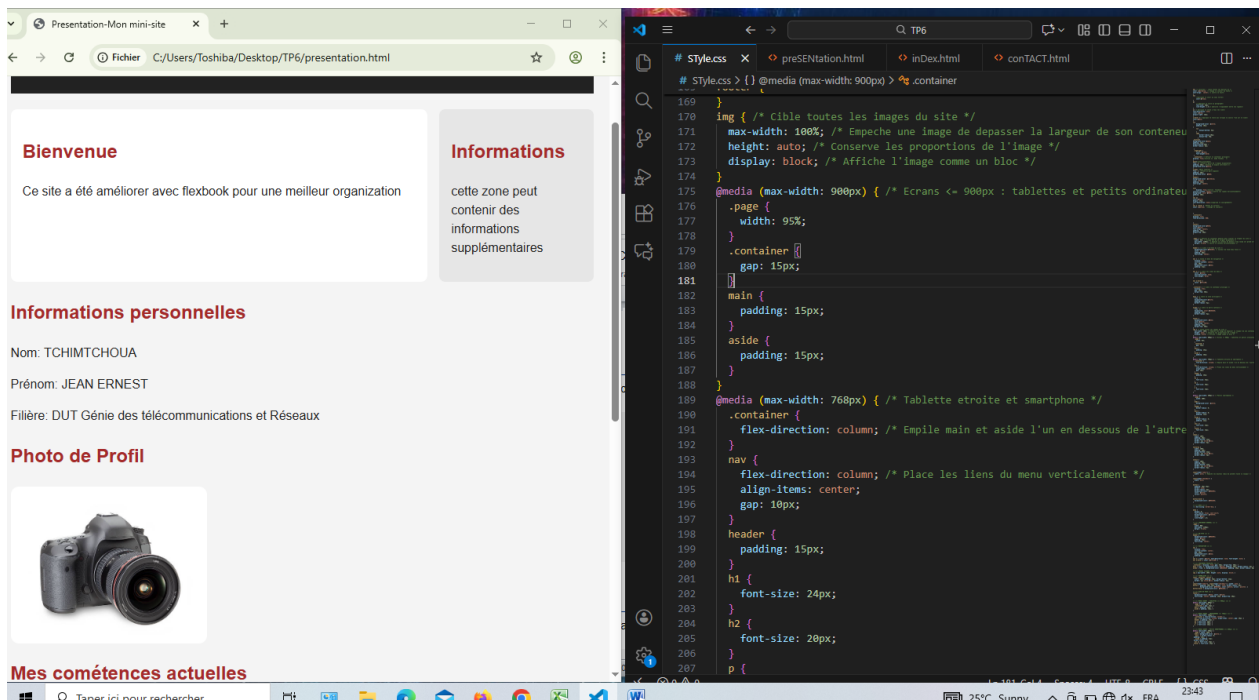
NB : Cette règle signifie que l'image peut diminuer si l'écran est petit, mais elle ne dépassera jamais son conteneur.



12. Première media query pour les tablettes

Une media query permet d'appliquer des styles uniquement lorsque certaines conditions sont vérifiées. Ajouter à la fin du fichier style.css :

```
@media (max-width: 900px) { /* Ecrans <= 900px : tablettes et petits ordinateurs */
  .page {
    width: 95%;
  }
  .container {
    gap: 15px;
  }
  main {
    padding: 15px;
  }
  aside {
    padding: 15px;
  }
}
```

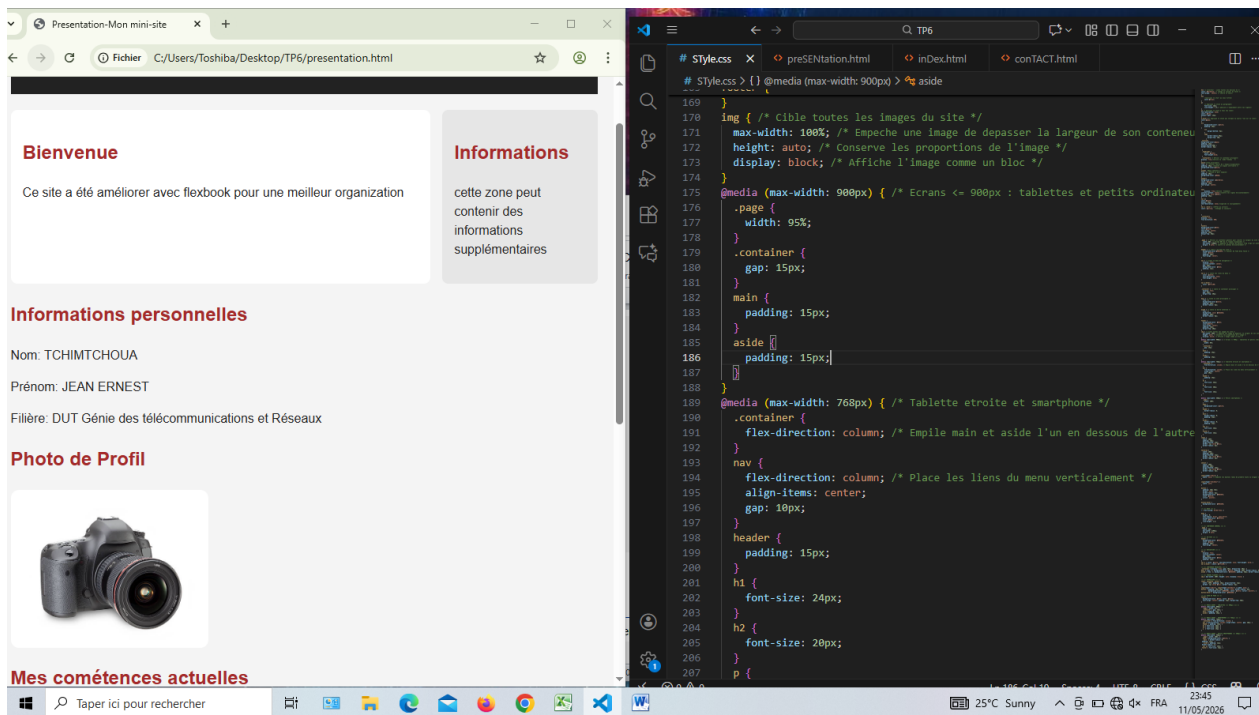


13. Deuxieme media query pour les smartphones

Sur smartphone, il n'est généralement pas confortable de garder main et aside cote a cote. Il faut les empiler. Ajouter :

```
@media (max-width: 768px) { /* Tablette étroite et smartphone */
  .container {
    flex-direction: column; /* Empile main et aside l'un en dessous de l'autre */
  }
  nav {
    flex-direction: column; /* Place les liens du menu verticalement */
    align-items: center;
    gap: 10px;
  }
  header {
    padding: 15px;
  }
  h1 {
    font-size: 24px;
  }
  h2 {
    font-size: 20px;
  }
  p {
    font-size: 15px;
  }
}
```

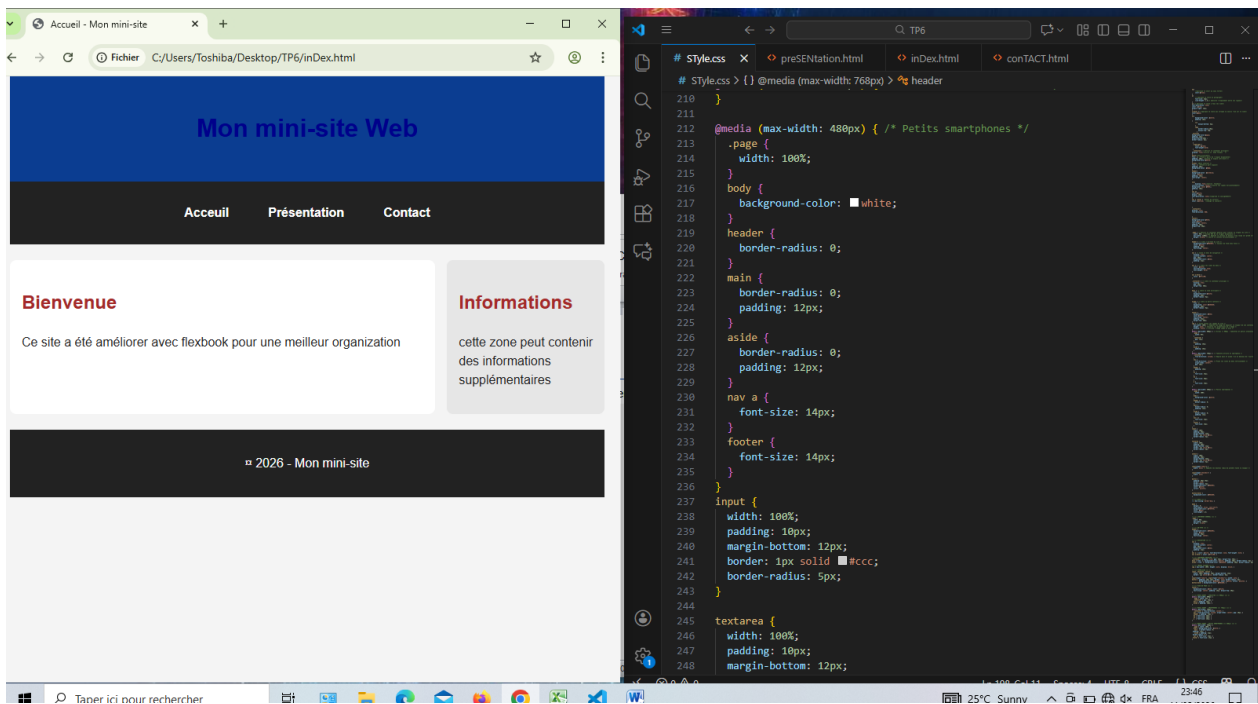
NB ; Cette étape est essentielle. Elle transforme une mise en page horizontale en mise en page verticale.



14. Troisième media query pour les très petits écrans

Certains smartphones ont une largeur très réduite. Il faut encore alléger les espacements. Ajouter :

```
@media (max-width: 480px) { /* Petits smartphones */
  .page {
    width: 100%;
  }
  body {
    background-color: white;
  }
  header {
    border-radius: 0;
  }
  main {
    border-radius: 0;
    padding: 12px;
  }
  aside {
    border-radius: 0;
    padding: 12px;
  }
  nav a {
    font-size: 14px;
  }
  footer {
    font-size: 14px;
  }
}
```

15. Adaptation du formulaire de contact

Le formulaire cree au TP4 doit aussi etre responsive. Ajouter dans style.css :

```
input {
  width: 100%;
  padding: 10px;
  margin-bottom: 12px;
  border: 1px solid #ccc;
  border-radius: 5px;
}

textarea {
  width: 100%;
  padding: 10px;
  margin-bottom: 12px;
  border: 1px solid #ccc;
  border-radius: 5px;
}

select {
  width: 100%;
  padding: 10px;
  margin-bottom: 12px;
  border: 1px solid #ccc;
  border-radius: 5px;
}

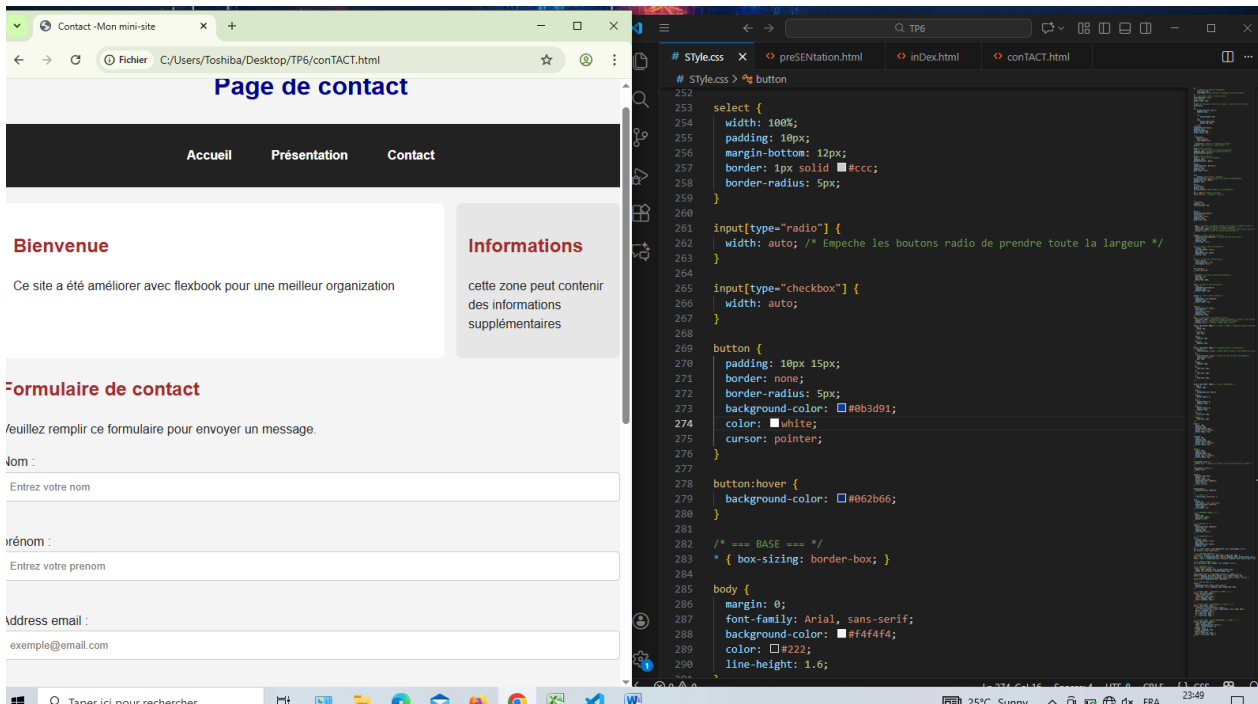
input[type="radio"] {
  width: auto; /* Empêche les boutons radio de prendre toute la largeur */
}

input[type="checkbox"] {
  width: auto;
}

button {
  padding: 10px 15px;
  border: none;
  border-radius: 5px;
}
```

```
background-color: #0b3d91;
color: white;
cursor: pointer;
}

button:hover {
background-color: #062b66;
}
```



16. Version complete recommandee du fichier CSS

Le fichier style.css peut maintenant contenir le code complet suivant (version finale recommandee) :

```
* { box-sizing: border-box; }

body {
margin: 0;
font-family: Arial, sans-serif;
background-color: #f4f4f4;
color: #222;
line-height: 1.6;
}

.page {
width: 90%;
max-width: 1100px;
margin: 0 auto;
}

header {
background-color: #0b3d91;
color: white;
padding: 20px;
text-align: center;
}
```

```

nav {
  display: flex;
  justify-content: center;
  gap: 20px;
  background-color: #222;
  padding: 12px;
}
nav a { color: white; text-decoration: none; font-weight: bold; }
nav a:hover { color: #ffcc00; }

/* === CONTENEUR PRINCIPAL === */
.container { display: flex; gap: 20px; margin-top: 20px; }
main { flex: 3; background-color: white; padding: 20px; border-radius: 8px; }
aside { flex: 1; background-color: #e6e6e6; padding: 20px; border-radius: 8px; }

img { max-width: 100%; height: auto; display: block; }

input, textarea, select {
  width: 100%; padding: 10px; margin-bottom: 12px;
  border: 1px solid #ccc; border-radius: 5px;
}
input[type="radio"], input[type="checkbox"] { width: auto; }
button { padding: 10px 15px; border: none; border-radius: 5px;
  background-color: #0b3d91; color: white; cursor: pointer; }
button:hover { background-color: #062b66; }

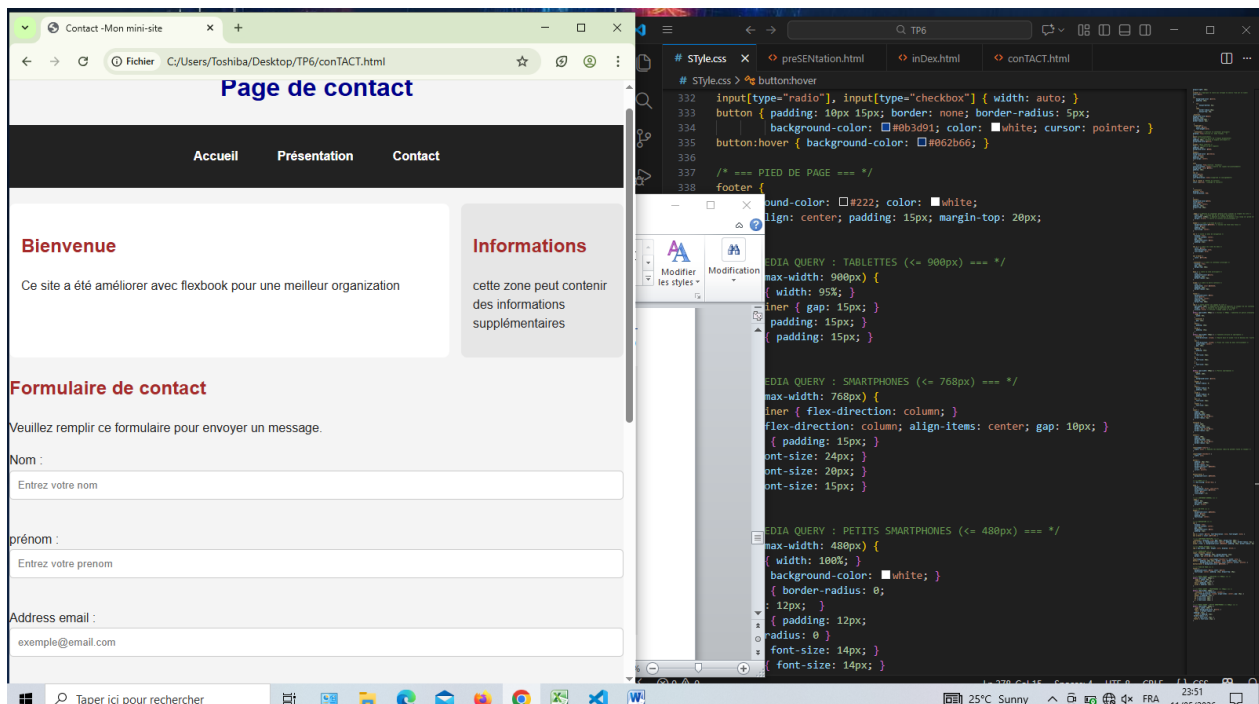
footer {
  background-color: #222; color: white;
  text-align: center; padding: 15px; margin-top: 20px;
}

/* === MEDIA QUERY : TABLETTES (<= 900px) === */
@media (max-width: 900px) {
  .page { width: 95%; }
  .container { gap: 15px; }
  main { padding: 15px; }
  aside { padding: 15px; }
}

/* === MEDIA QUERY : SMARTPHONES (<= 768px) === */
@media (max-width: 768px) {
  .container { flex-direction: column; }
  nav { flex-direction: column; align-items: center; gap: 10px; }
  header { padding: 15px; }
  h1 { font-size: 24px; }
  h2 { font-size: 20px; }
  p { font-size: 15px; }
}

/* === MEDIA QUERY : PETITS SMARTPHONES (<= 480px) === */
@media (max-width: 480px) {
  .page { width: 100%; }
  body { background-color: white; }
  main, { border-radius: 0;
padding: 12px; }
  aside { padding: 12px;
border-radius: 0 }
  nav a { font-size: 14px; }
  footer { font-size: 14px; }
}

```



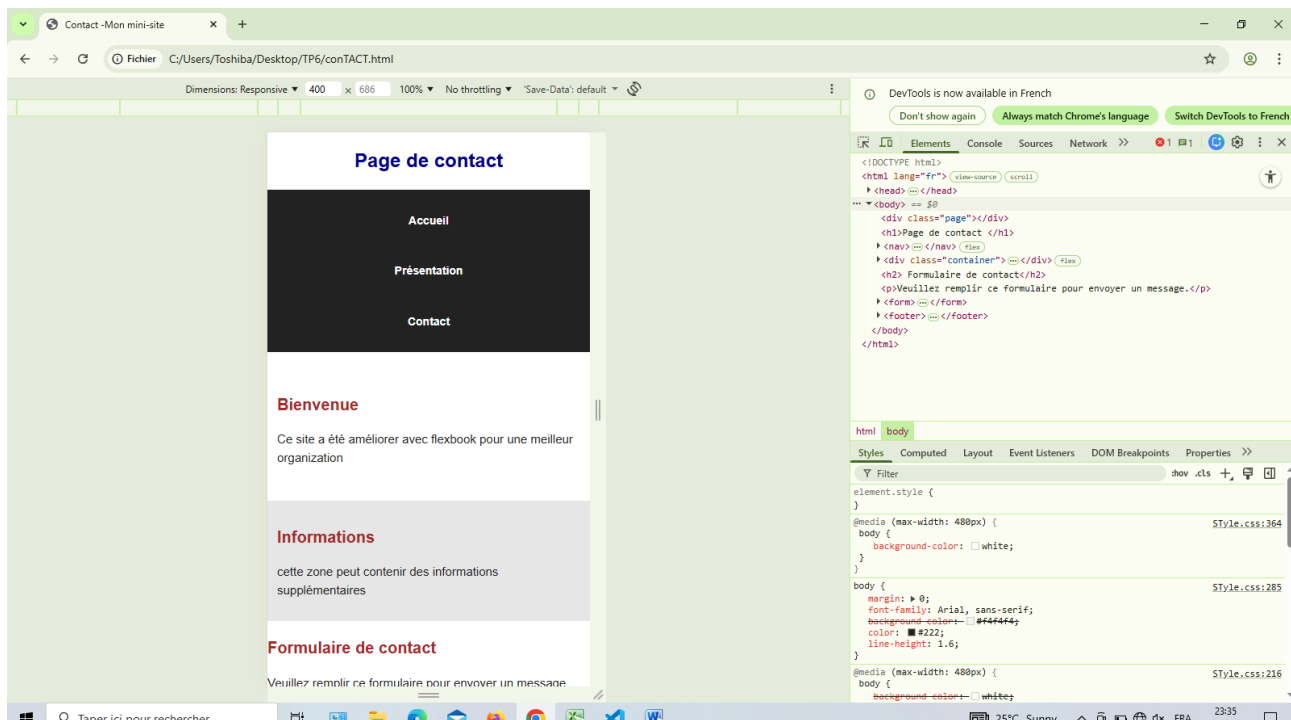
17. Test avec les outils de developpement du navigateur

Pour verifier le responsive design, nous avons ouvert le site dans Google Chrome ou Mozilla Firefox ensuite :

Appuyer sur : F12, puis activer le mode mobile avec : Ctrl + Shift + M

Dans ce mode, tester plusieurs tailles d'ecran : ordinateur, tablette, smartphone, petit smartphone.

Observer particulierement : le menu, l'image, la zone principale, la barre laterale, le formulaire et l'absence de defilement horizontal.



18. Travail demande a l'etudiant

Le travail que nous avons réaliser est le suivant :

- ajouter la balise meta viewport dans toutes les pages HTML
- entourer le contenu de chaque page avec le conteneur .page
- conserver une structure coherente avec header, nav, .container, main, aside et footer
- rendre les images fluides
- adapter le formulaire de contact
- ajouter les media queries a 900px, 768px et 480px
- tester le site avec les outils developpeur
- verifier que les trois pages restent lisibles sur mobile

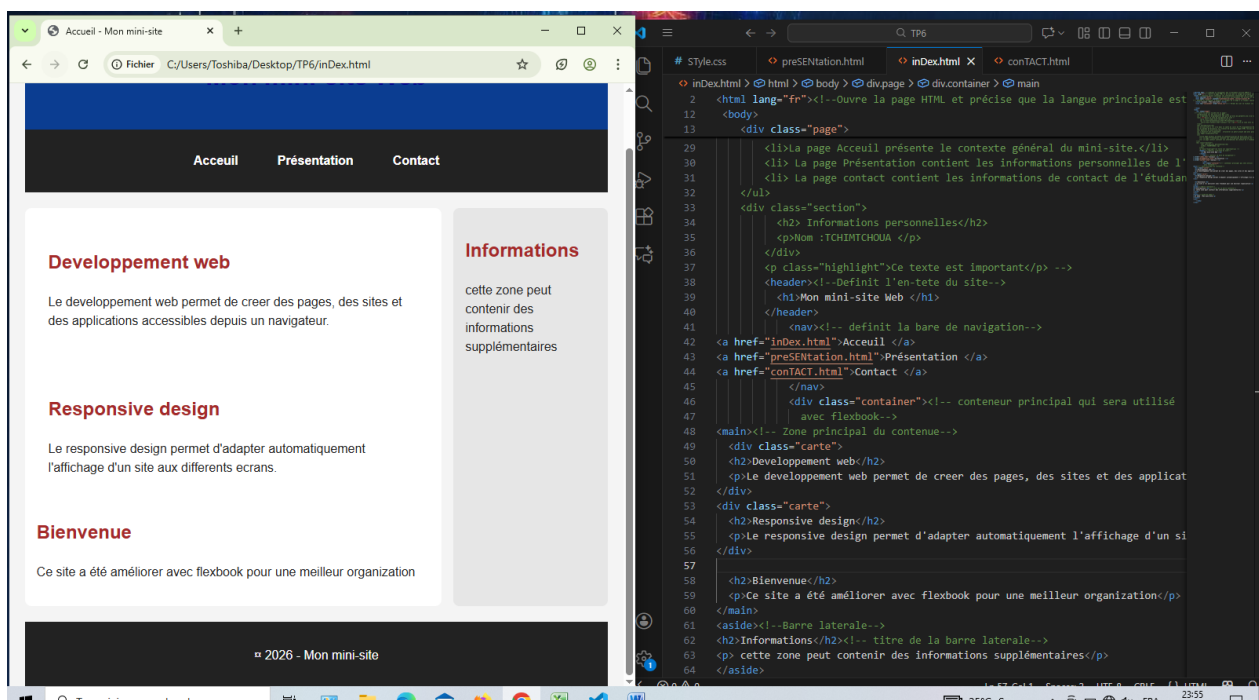
19. Mini-exercice d'application : la classe .carte

Ajouter une classe CSS appelee .carte dans style.css :

```
.carte { /* Cible chaque carte d'information */  
  background-color: white;  
  padding: 15px;  
  border-radius: 8px;  
  margin-bottom: 15px;  
}
```

Dans index.html, ajouter dans la zone main :

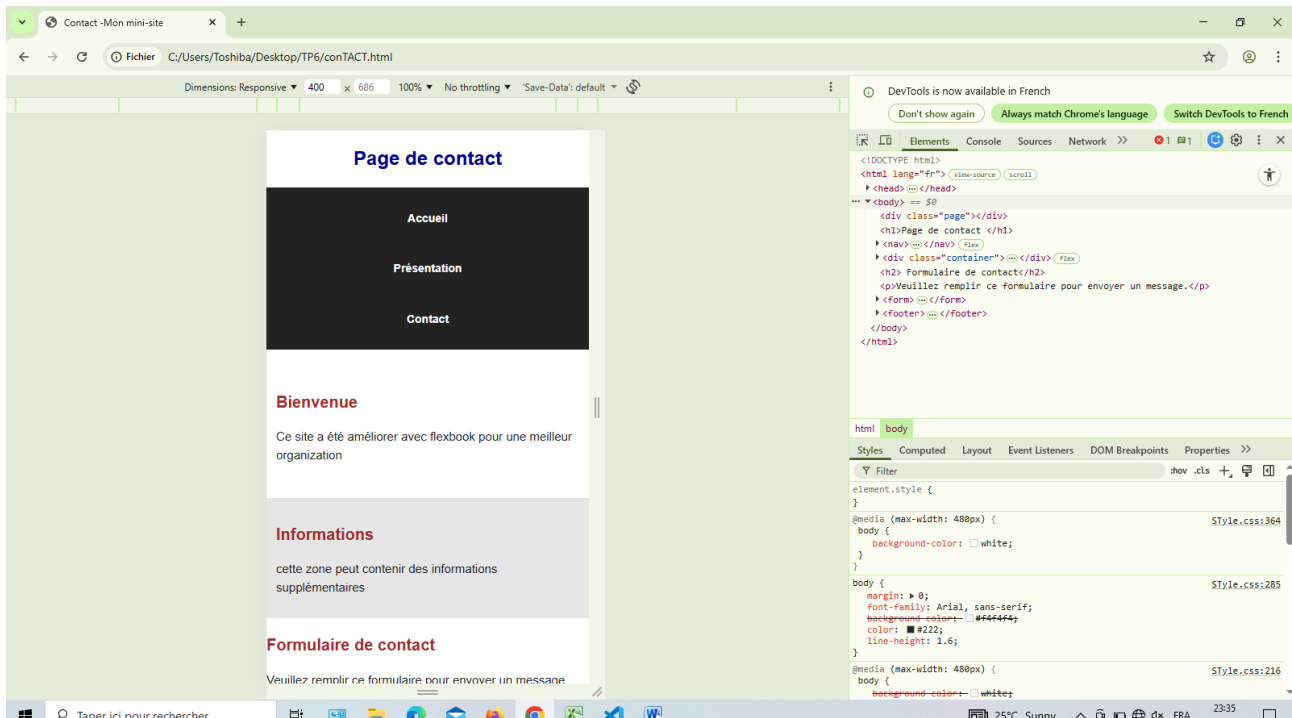
```
<div class="carte">  
  <h2>Developpement web</h2>  
  <p>Le developpement web permet de creer des pages, des sites et des applications  
  accessibles depuis un navigateur.</p>  
</div>  
<div class="carte">  
  <h2>Responsive design</h2>  
  <p>Le responsive design permet d'adapter automatiquement l'affichage d'un site aux  
  differents ecrans.</p>  
</div>
```



20. Resultat attendu a la fin du TP

A la fin du TP6, le mini-site doit être consultable correctement sur ordinateur, tablette et smartphone.

- sur ordinateur : le contenu principal et la barre latérale sont côte à côte
- sur tablette étroite ou smartphone : les blocs sont empilés verticalement
- le menu horizontal devient vertical sur mobile
- les images ne dépassent jamais la largeur de l'écran
- le formulaire reste lisible et utilisable
- les textes restent confortables à lire
- aucun défilement horizontal gênant ne doit apparaître



21. Livrable attendu

Le dossier à remettre doit être nommé : TP6_Nom_Prenom

Il doit contenir :

- index.html
- presentation.html
- contact.html
- style.css
- le dossier images

Le site doit fonctionner en ouvrant simplement le fichier : index.html

TP6 Rechercher dans : TP6		
Nom	Modifié le	Type
images	11/05/2026 16:47	Do
conTACT.html	11/05/2026 23:30	Ch
inDex.html	11/05/2026 23:55	Ch
preSEnTation.html	11/05/2026 23:29	Ch
profil.jpg	10/05/2025 08:09	Fic
SStyle.css	11/05/2026 23:54	Fic
Sélectionnez un fichier à afficher.		